Ước lượng cao độ F_0 kiểu Praat trên DE10-Standard

Từ file WAV 16 kHz qua HPS, FPGA và trở về HPS, so với Praat 7.0.03 trên laptop
(Intel Quartus Prime — hai đường: IP Catalog và HDL tự viết)

Bản v3 — viết lại theo đề cương chốt v2 (10/11/2026)

Mục lục

Chương 0. Bài toán, mục tiêu so sánh, và Praat thật sự làm gì

▶ MỤC TIÊU HỌC TẬP

- Nắm vững bản chất bài toán ước lượng tần số cơ bản (F_0) và lý do vì sao hai phương pháp trực giác (đếm điểm qua không và tìm đỉnh phổ FFT) thất bại trên tín hiệu tiếng nói thực tế.
- Hiểu rõ nguyên lý cốt lõi của Boersma: chuẩn hóa hàm tự tương quan bằng cách chia cho hàm tự tương quan của cửa sổ Hanning để triệt tiêu độ suy giảm giả tạo ở các độ trễ lớn.
- Nắm tường tận 8 bước thuật toán trong mã nguồn Praat 7.0.03 cùng 10 tham số chuẩn, đối chiếu chính xác giữa các khâu phần mềm của Praat và các khối xử lý phần cứng trên FPGA.
- Hiểu rõ kiến trúc toàn hệ thống: luồng dữ liệu từ tệp WAV qua HPS, cầu nối Lightweight HPS-to-FPGA, lõi tính toán trên FPGA, và đường trả kết quả F0 về HPS để ghi tệp nhị phân.
- Nắm chắc định nghĩa 4 chỉ số sai số chuẩn trong so sánh cao độ: VDE, GPE, FFE và sai số cent mịn, cùng cách tính tay trên từng khung.
- Phân tích ngân sách chu kỳ từ nguyên lý đầu tiên: 50 MHz, 3.125 chu kỳ/mẫu, 500.000 chu kỳ/bước nhảy 10 ms; dẫn xuất chính xác 98.540 phép nhân-cộng MAC và đo thực tế 98.543 chu kỳ/khung, từ đó xác lập hệ số thời gian thực (RTF).
- Hiểu kiến trúc kiểm chứng ba tầng và vai trò của bộ mô hình tương đương (twin) số nguyên `golden_v2/fixed_twin.py` làm đặc tả phần cứng cho toàn bộ dự án.

0.1. Một file, hai câu trả lời

Một tệp âm thanh WAV chuẩn 16 kHz, kênh đơn (mono), định dạng mẫu PCM 16-bit có dấu được đưa vào hệ thống. Trên máy tính cá nhân, phần mềm Praat đọc tệp âm thanh này và xuất ra một đường cao độ F_0 với chu kỳ phân tích 10 ms. Trên bo mạch DE10-Standard, hệ thống vi xử lý HPS (Hard Processor System, Cortex-A9 lõi kép) đọc cùng tệp WAV đó từ thẻ nhớ SD, truyền các mẫu âm thanh qua cầu nối bus sang vùng logic FPGA (Cyclone V 5CSXFC6D6F31C6N). Lõi phần cứng trên FPGA tính toán cao độ từng khung và trả kết quả về HPS dưới dạng tệp nhị phân `pitch_f0.bin` ở định dạng số cố định Q12.4.

Câu hỏi trung tâm mà anh Tú cùng bất kỳ kỹ sư phần cứng nào cũng phải đối mặt là: hai đường cong cao độ đó lệch nhau bao nhiêu, và nguyên nhân vật lý hay số học nào tạo ra sự chênh lệch ở từng mili-giây? Để trả lời thấu đáo câu hỏi này, trước hết ta phải hiểu vì sao việc ước lượng cao độ tiếng nói lại khó, và vì sao những cách tiếp cận trực giác thông thường đều sụp đổ khi áp dụng vào thực tế.

Sự sụp đổ của phương pháp đếm điểm qua không (Zero-Crossing Counting)

Phương pháp đơn giản nhất mà người mới học xử lý tín hiệu thường nghĩ đến là đếm số lần tín hiệu đổi dấu qua mức 0 (zero-crossing) trong một khoảng thời gian, từ đó suy ra tần số. Tuy nhiên, một sự thật vật lý cơ bản là: tiếng nói có thanh (voiced speech) của con người hầu như không bao giờ là một sóng hình sin đơn âm. Khi dây thanh âm rung động, luồng khí đóng mở theo chu kỳ tạo ra dạng sóng có chứa rất nhiều hài âm (harmonics). Nếu hài âm bậc hai ($2f_0$) có biên độ mạnh và lệch pha so với sóng cơ bản, nó sẽ uốn dạng sóng tổng hợp cắt qua trục 0 nhiều lần trong cùng một chu kỳ lặp lại.

Để thấy rõ sự thất bại này bằng số liệu định lượng, ta xét một đoạn tín hiệu tổng hợp được tạo bởi kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`:

$$x(t) = \sin(2\pi \cdot 100t) - 0{,}75\cos(2\pi \cdot 200t) \tag{1}$$

Tần số cơ bản thực sự là $f_0 = 100{,}0$ Hz, tần số lấy mẫu $f_s = 16.000$ Hz. Một chu kỳ cơ bản kéo dài $T_0 = 1/100 = 10$ ms, tương ứng đúng $P = 160$ mẫu. Nếu tín hiệu là sóng sin thuần, một chu kỳ 160 mẫu sẽ cắt qua trục 0 đúng 2 lần. Nhưng khi kiểm tra trên đoạn tín hiệu dài 3 chu kỳ (480 mẫu), bộ đếm điểm qua không ghi nhận tổng cộng 12 lần đổi dấu, tức trung bình 4 điểm qua không trong mỗi chu kỳ. Tần số ước lượng bị thổi phồng thành:

$$f_{\text{est}} = \frac{12}{2 \times 3 \times 0{,}01\ \text{s}} = 200{,}0\ \text{Hz} \tag{2}$$

Thuật toán bị đánh lừa hoàn toàn bởi hài âm bậc hai, báo sai lệch cao độ gấp đôi (100% lỗi) ngay trên một tín hiệu hoàn toàn tuần hoàn và không hề có nhiễu bên ngoài.

Ví dụ tính tay thu nhỏ. Để nhìn thấy từng bước ở mức mẫu, xét 16 mẫu rời rạc đại diện cho một chu kỳ rút gọn (lấy trực tiếp từ log của `verify_v3/ch00_naive_methods.py`):

n	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$x[n]$	0	-18	-9	36	100	149	151	95	0	-95	-151	-149	-100	-36	9	18

Quan sát dãy mẫu: từ $n = 2$ (giá trị -9) sang $n = 3$ ($+36$), tín hiệu đổi dấu lần thứ nhất. Đến $n = 8$, tín hiệu bằng 0 và giảm xuống -95 ở $n = 9$, đổi dấu lần thứ hai. Từ $n = 13$ (-36) sang $n = 14$ ($+9$), tín hiệu lại đổi dấu lần thứ ba. Cùng với điểm 0 ban đầu ở $n = 0$, ta có 4 lần cắt qua 0 trong chỉ một chu kỳ cơ bản. Mạch đếm phần cứng nếu dựa vào hiện tượng này sẽ nhận diện nhầm chu kỳ thành 8 mẫu thay vì 16 mẫu.

Sự sụp đổ của phương pháp chọn đỉnh phổ FFT (FFT Peak Picking)

Cách tiếp cận thứ hai là chuyển tín hiệu sang miền tần số bằng phép biến đổi Fourier nhanh (FFT, Fast Fourier Transform) rồi tìm vị trí bin có biên độ cực đại. Để bám sát ngữ điệu biến đổi liên tục của tiếng nói, ta bắt buộc phải phân tích trên từng khung thời gian ngắn khoảng vài chục mili-giây (thường từ 20 ms đến 40 ms). Tại tần số lấy mẫu $f_s = 16.000$ Hz, một khung 40 ms tương ứng với 640 mẫu (hoặc 638 mẫu theo quy ước làm tròn đối xứng của Praat).

Độ phân giải tần số của phép phân tích FFT bị chặn cứng bởi chiều dài khung N:

$$\Delta f = \frac{f_s}{N} = \frac{16.000}{640} = 25{,}00 \text{ Hz} \quad (\text{với } N = 638, \Delta f = \frac{16.000}{638} \approx 25{,}08 \text{ Hz}) \tag{3}$$

Khoảng cách giữa hai vạch phổ kề nhau lên tới 25 Hz. Hãy xét một giọng nam trầm có tần số cơ bản thực là $f_0 = 112{,}5$ Hz. Tần số này rơi vào chính giữa bin 4 ($4 \times 25 = 100$ Hz) và bin 5 ($5 \times 25 = 125$ Hz). Bộ chọn đỉnh phổ dù hoàn hảo nhất cũng chỉ có thể chỉ định bin 4 hoặc bin 5, gây ra sai số tuyệt đối ít nhất là $12{,}5$ Hz.

Trong âm nhạc học và ngữ âm học, tai người cảm nhận cao độ theo hàm logarit của tần số. Đơn vị đo độ chênh lệch cao độ chuẩn là **cent**, trong đó 1 quãng tám (tần số gấp đôi) tương ứng với 1.200 cent, và 1 nửa cung (semitone) tương ứng với 100 cent:

$$\Delta\text{cent} = 1.200 \cdot \left| \log_2 \left(\frac{f_{\text{est}}}{f_0} \right) \right| \tag{4}$$

Áp dụng cho trường hợp chọn nhầm bin 100 Hz khi tần số thực là 112,5 Hz:

$$\Delta\text{cent} = 1.200 \cdot \left| \log_2 \left(\frac{100{,}0}{112{,}5} \right) \right| \approx 203{,}91 \text{ cent} \tag{5}$$

(kết quả được kiểm chứng trong `verify_v3/ch00_naive_methods.log`). Sai số hơn 200 cent tương đương với hai nửa cung (một cung nguyên) trong âm nhạc. Một bộ ước lượng F_0 với sai số thô như vậy hoàn toàn vô giá trị trong các ứng dụng phân tích ngữ điệu hoặc đối sánh khoa học. Muốn giảm Δf xuống dưới 1 Hz, FFT cần cửa sổ dài hơn 1 giây, nhưng trong 1 giây đó người nói đã phát âm qua 3 đến 5 âm tiết khác nhau, tính dừng của tín hiệu bị phá vỡ hoàn toàn.

Bản chất của phép tự tương quan (Autocorrelation)

Để khắc phục triệt để hai nhược điểm trên, ta quay về nguyên lý đầu tiên: **một tín hiệu tuần hoàn luôn có hình dạng giống hệt chính nó khi bị dịch trễ đi đúng một chu kỳ cơ bản**. Phép đo mức độ tương đồng giữa tín hiệu $x[n]$ và bản sao dịch trễ của nó $x[n + \tau]$ ở độ trễ τ (lag) được định nghĩa qua hàm tự tương quan ngắn hạn:

$$R[\tau] = \sum_{n=0}^{N-1-\tau} x[n]x[n+\tau] \tag{6}$$

Ví dụ tính tay số nguyên thu nhỏ. Xét một tín hiệu rời rạc tuần hoàn hoàn hảo với chu kỳ cơ bản $P = 3$ mẫu, độ dài khung $N = 9$ mẫu:

$$x = [3, -1, -2,\ 3, -1, -2,\ 3, -1, -2] \tag{7}$$

Ta tính trực tiếp giá trị tích vô hướng $R[\tau]$ theo từng bước cho các độ trễ từ $\tau = 0$ đến $\tau = 5$ (trích xuất từ kịch bản kiểm chứng `verify_v3/ch00_naive_methods.py`):

- Tại $\tau = 0$: lấy 9 cặp mẫu tự nhân với chính nó:

$$\begin{aligned} R[0] &= 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 + 3^2 + (-1)^2 + (-2)^2 \\ &= 9 + 1 + 4 + 9 + 1 + 4 + 9 + 1 + 4 = 42 \end{aligned}$$

- Tại $\tau = 1$: nhân 8 cặp mẫu kề nhau:

$$\begin{aligned} R[1] &= 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) + (-2)3 + 3(-1) + (-1)(-2) \\ &= -3 + 2 - 6 - 3 + 2 - 6 - 3 + 2 = -15 \end{aligned}$$

- Tại $\tau = 2$: nhân 7 cặp mẫu cách nhau 2 vị trí:

$$\begin{aligned} R[2] &= 3(-2) + (-1)3 + (-2)(-1) + 3(-2) + (-1)3 + (-2)(-1) + 3(-2) \\ &= -6 - 3 + 2 - 6 - 3 + 2 - 6 = -20 \end{aligned}$$

- Tại $\tau = 3$: nhân 6 cặp mẫu cách nhau đúng một chu kỳ $P = 3$:

$$\begin{aligned} R[3] &= 3(3) + (-1)(-1) + (-2)(-2) + 3(3) + (-1)(-1) + (-2)(-2) \\ &= 9 + 1 + 4 + 9 + 1 + 4 = 28 \end{aligned}$$

- Tại $\tau = 4$: nhân 5 cặp mẫu, thu được $R[4] = -8$.
- Tại $\tau = 5$: nhân 4 cặp mẫu, thu được $R[5] = -13$.

Chỗ dễ vướng: Vì sao $R[3] = 28$ lại bé hơn $R[0] = 42$ dù tín hiệu tuần hoàn tuyệt đối? Đây là bẫy tư duy kinh điển mà người thiết kế phần cứng hay gặp. Ở độ trễ $\tau = 3$, số cặp mẫu còn lại để nhân với nhau chỉ là $N - \tau = 9 - 3 = 6$ cặp, trong khi ở $\tau = 0$ ta cộng dồn tới 9 cặp. Nếu ta tính giá trị trung bình không chệch bằng cách chia tổng tích lũy cho số cặp mẫu tham gia:

$$\frac{R[0]}{9-0} = \frac{42}{9} \approx 4{,}67 \tag{8}$$

$$\frac{R[3]}{9-3} = \frac{28}{6} \approx 4{,}67 \tag{9}$$

Hai con số hoàn toàn trùng khớp! Đỉnh tương quan tại $\tau = 3$ đã chỉ ra chính xác chu kỳ cơ bản của tín hiệu là 3 mẫu mà không bị nhiễu hài âm đánh lừa như phương pháp đếm điểm qua 0, cũng không bị lượng tử hóa thô thiển như phương pháp chọn đỉnh FFT.

Tóm lại: Đếm điểm qua không thất bại vì hài âm bậc hai tạo thêm điểm cắt giả, lấy đỉnh FFT thất bại vì độ phân giải bin thô tới 204 cent ở 100 Hz; phương pháp tự tương quan đo mức độ tương đồng theo miền thời gian, mở ra con đường xác định chu kỳ cơ bản chính xác và bền vững trước cấu trúc hài âm phức tạp của tiếng nói.